Ripasso completo: Arduino e linguaggio C

Approfondimento della lezione 1 — 2MAK, SST-INF-01c — Tutto il firmware dell'anno scorso

15/10/2026

Nota

Questa dispensa raccoglie **tutto il firmware** dell'anno scorso, con più dettagli ed esempi della dispensa A01 (che resta il riassunto della lezione del 15/10). Serve a chi deve **ripassare** e a chi è **arrivato quest'anno**. È la base della **verifica di ripasso** del **29 ottobre**. Fonti: la dispensa Arduino usata l'anno scorso, le dispense di C e di firmware, e la prima ora della lezione del 15/10.

1 La scheda Arduino Uno

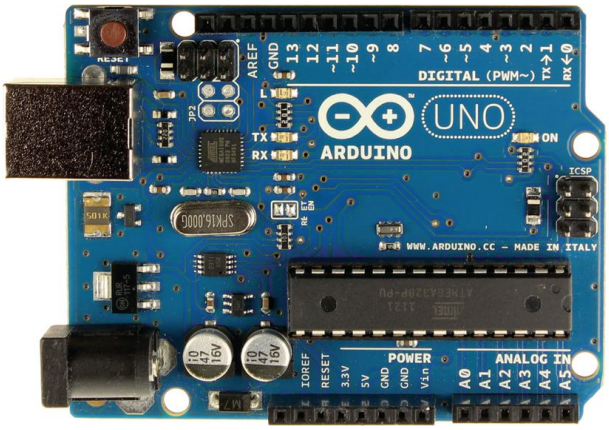


Figura 1: La scheda Arduino Uno: a sinistra il connettore USB e la presa di alimentazione, in alto i pin digitali 0–13, in basso i pin di alimentazione (5V, GND) e gli ingressi analogici A0–A5.

- **Connettore USB**: ha un doppio scopo. Serve ad **alimentare** la scheda con i 5 V della porta USB del computer, e a **scambiare dati** con Arduino (molto utile per capire che cosa succede a bordo, con il monitor seriale).
- **Presa di alimentazione**: in un angolo c'è un piccolo barilotto nero, in cui si infila uno spinotto (*jack*) con foro interno da 2,1 mm. La tensione che arriva viene portata a 5 V da un **regolatore di tensione** sulla scheda. Spesso si collega una **batteria da 9 V** con una clip dotata di jack, per usare Arduino lontano dal computer.
- **GND** (*ground*, "terra" o "massa"): è il **polo negativo** dell'alimentazione, chiamato anche "comune" o "0 volt". Sulla scheda ci sono più punti GND a cui collegarsi.
- **5V**: questo pin fornisce la tensione di 5 V **regolata e stabilizzata** da Arduino, per alimentare piccoli circuiti esterni.

- **Pin da 0 a 13**: 14 pin **digitali**, che si possono configurare via software come **ingressi** o **uscite**. Arduino è un dispositivo digitale: questi pin generano o leggono solo un livello **alto** (5 V) o **basso** (0 V). I pin **0 e 1** sono segnati anche **RX** e **TX**: sono collegati alla **porta seriale** del chip (quella usata anche dall'USB), quindi è meglio lasciarli liberi.
- **A0–A5**: sei pin in grado di leggere **livelli analogici** (una tensione qualsiasi tra 0 e 5 V) e convertirli in un numero da usare nel programma.

1.1 Il cervello: il microcontrollore ATmega328

La maggior parte delle schede Arduino usa microcontrollori della famiglia **ATmega**, progettati da Atmel (oggi parte di **Microchip**). Arduino Uno usa l'**ATmega328**. Le caratteristiche che si guardano per scegliere un microcontrollore:

Caratteristica	Che cos'è	ATmega328 (Arduino Uno)
Velocità	frequenza di lavoro massima	16 MHz
RAM	memoria "volatile" per calcoli e variabili	2 KB
Flash	memoria in cui è salvato il programma	32 KB
EEPROM	memoria "permanente" per parametri	1 KB
GPIO	pin usabili come ingressi/uscite digitali	14
Pin analogici	ingressi con convertitore analogico-digitale	6
PWM	pin che generano un'onda quadra regolabile	6 (segnati con ~)
Porte seriali	linee per scambiare dati	1
Alimentazione	tensioni ammesse per il chip	da 1,8 a 5,5 V

Nota

La **EEPROM** ha un numero limitato di scritture: si usa per configurazioni che cambiano raramente. Il **PWM** (*Pulse Width Modulation*) serve per esempio a regolare la velocità di un motore o la luminosità di un LED. L'ATmega328 esiste anche nel contenitore **DIL** (dual in line), che si può montare su una breadboard: con pochi altri componenti si costruisce una scheda a microcontrollore personalizzata. Per i prodotti finiti si usano spesso gli **ATtiny**, i "fratelli minori", che possono costare meno di un euro.

2 La struttura di un programma

Definizione

Un programma Arduino (*sketch*) ha due **funzioni** che non possono mai mancare:

- **`setup()`**: eseguita **una volta sola**, all'avvio o quando Arduino viene alimentato. Contiene le **configurazioni iniziali** (direzione dei pin, velocità della seriale, ...);
- **`loop()`**: eseguita **ciclicamente, all'infinito**, dopo `setup()`. Contiene il lavoro vero e proprio.

```
void setup()
{
  // codice eseguito una volta sola
}

void loop()
{
  // codice ripetuto all'infinito
}
```

Nota

Quando si programma in C **fuori da Arduino** (per esempio sul computer), l'unica funzione obbligatoria è **`main()`**: è da lì che parte il programma. Arduino "nasconde" il `main` e chiama lui `setup` e `loop`. Che cos'è una funzione, e come si scrivono le proprie, è spiegato nella dispensa A01.

Attenzione

Regole di scrittura del C che valgono sempre:

- ogni istruzione finisce con il **punto e virgola** `;`;
- le **parentesi graffe** `{ }` racchiudono un blocco di istruzioni (il corpo di una funzione, di un `if`, di un ciclo);
- i **commenti** non vengono eseguiti: `//` commenta fino alla fine della riga, `/* ... */` commenta tutto quello che sta in mezzo, anche su più righe;
- il C distingue **maiuscole e minuscole**: `digitalWrite` è giusto, `digitalwrite` è un errore.

3 Bit, byte e numeri binari

Definizione

Il **bit** è la cifra binaria: vale **0** oppure **1**. Un **byte** è fatto da **8 bit**. Con n bit si rappresentano 2^n valori diversi: con 8 bit, $2^8 = 256$ valori.

Esempio

Da binario a decimale: ogni cifra si moltiplica per il suo **peso**, che è una potenza di 2 (da destra: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, …). Per esempio:

$$10010_2 = 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 18$$

Da decimale a binario: si divide ripetutamente per 2 e si scrivono i resti, poi si leggono **dall'ultimo al primo**. Per 18: $18 : 2 = 9$ resto 0; $9 : 2 = 4$ resto 1; $4 : 2 = 2$ resto 0; $2 : 2 = 1$ resto 0; $1 : 2 = 0$ resto 1. Letti al contrario: 10010.

4 Le variabili e i tipi di dato

Definizione

Una **variabile** è un **contenitore** con un nome, in cui il programma memorizza temporaneamente un valore da usare nei calcoli. Come i **cassetti di un armadio**, le variabili hanno **dimensioni diverse** a seconda dei numeri che devono contenere: per questo ogni variabile ha un **tipo**.

Tipo (Arduino Uno)	Byte	Valori
`char`	1	da -128 a 127
`byte` (o `unsigned char`)	1	da 0 a 255
`int`	2	da $-32\,768$ a $32\,767$
`unsigned int`	2	da 0 a $65\,535$
`long`	4	da circa $-2{,}1$ a $+2{,}1$ miliardi
`float` (e `double`)	4	numeri con la virgola, circa 6–7 cifre significative

- Il **`char`** contiene anche un **carattere**: i caratteri corrispondono a numeri secondo il **codice ASCII**. Le due variabili qui sotto hanno lo **stesso valore**, perché la `A` maiuscola in ASCII è il 65:

```
char c1 = 'A';
char c2 = 65;
```

  Un carattere si scrive tra **virgolette singole** (`'A'`); il testo tra **virgolette doppie** (`"ciao"`).

- **`unsigned`** (senza segno) significa che la variabile non contiene numeri negativi: con lo stesso spazio arriva a valori positivi doppi.
- Su Arduino Uno `float` e `double` sono **la stessa cosa**: 4 byte.

Attenzione

La **dimensione** di un tipo **dipende dalla macchina**: su Arduino Uno `int` occupa 2 byte, sul computer di solito 4. E attenzione all'intervallo del `char`: è **con segno** ($-128 \ldots 127$); da 0 a 255 è il tipo `byte`. Alla domanda "quanto è grande un `char`?" la risposta è **1 byte**, cioè 8 bit.

4.1 Dichiarare e assegnare

```
int temperatura;        // dichiarazione: creo la variabile
temperatura = 21;       // assegnazione: ci metto dentro 21
int conteggio = 0;      // dichiarazione e assegnazione insieme
```

Nota

I **nomi** delle variabili: lettere, cifre e trattino basso, senza spazi, **non possono iniziare** con una **cifra** e non possono essere **parole riservate** del C (come `int`, `if`, `for`, `while`, `void`, `return`). Sceglieteli in modo che dicano cosa contengono: `temperatura` è meglio di `t`.

4.2 Le costanti con `#define`

Un valore che non cambia mai si può chiamare con un nome, con **`#define`** (senza = e senza punto e virgola). Per convenzione il nome si scrive **in maiuscolo**:

```
#define LED_ROSSO 3
#define BASE 7
```

Da quel punto in poi il programma può usare `LED_ROSSO` al posto di 3. Tutti i dettagli sono nella dispensa A01.

4.3 Gli array

Definizione

Un **array** (vettore) è una sequenza di valori **dello stesso tipo**, sotto un unico nome. Ogni elemento si raggiunge con un **indice** tra parentesi quadre. L'indice **parte da 0**: in un array di 5 elementi gli indici vanno da 0 a 4.

```
int valori[2];            // array di due elementi
valori[0] = 42;           // imposta il primo elemento
valori[1] = -42;          // imposta il secondo elemento

int altri[] = { 42, -42 }; // stesso contenuto, scritto tutto insieme
int primo = altri[0];      // primo vale 42
```

Attenzione

- L'ultimo indice valido è **dimensione** − 1: in `int v[5]` non esiste `v[5]`. Il compilatore **non segnala** l'errore: il programma legge o scrive memoria che non gli appartiene, con risultati imprevedibili.
- Un array dichiarato **dentro una funzione** e non inizializzato contiene **valori casuali**: inizializzatelo sempre (per esempio `int v[10] = {0};` lo mette tutto a zero).

5 Gli operatori

Definizione

Gli **operatori** sono i simboli che mettono in relazione i valori. Ne abbiamo visti quattro famiglie: **assegnazione**, **matematici**, **di confronto** e **logici**.

5.1 Assegnazione: =

Il simbolo **=** **copia** il valore che sta a destra nella variabile che sta a sinistra. Non è una domanda ("sono uguali?"): è un **ordine**. Va letto "prende": `risultato = a + b` si legge "risultato prende a più b".

```
char risultato;
char a;
char b;
a = 5;
b = 6;
risultato = a + b;   // risultato vale 11
```

5.2 Operatori matematici

+	somma
-	sottrazione
*	moltiplicazione
/	divisione
%	modulo (resto della divisione)

Definizione

Tra **numeri interi** la divisione `/` dà solo il **quoziente intero**: la parte dopo la virgola viene buttata. Il **modulo** `%`, che **non c'entra** con la percentuale, dà il **resto** della divisione:

$$5 / 2 = 2 \qquad 5 \% 2 = 1$$

Esempio

```
int a = 10;
int b = 2;
int c = a + b;   // 12
int d = a - b;   // 8
int e = a / b;   // 5
int f = a * b;   // 20
int g = 7 % 3;   // 1 (7 diviso 3 fa 2 con resto 1)
```

Il modulo è utilissimo: `n % 2` vale 0 se `n` è **pari**, 1 se è **dispari**.

5.3 Operatori di confronto

Confrontano due valori. Il risultato è sempre **vero** oppure **falso** (in C: 1 oppure 0).

Simbolo	Significato	Esempio
>	maggiore	`5 > 3` è vero
<	minore	`7 < 3` è falso
>=	maggiore o uguale	`5 >= 5` è vero
<=	minore o uguale	`2 <= 1` è falso
==	uguale	`5 == 5` è vero
!=	diverso	`9 != 3` è vero

Il risultato di un confronto si può anche mettere in una variabile: dopo `char a = 5 > 3;` la variabile `a` vale **vero** (1); dopo `char r = (7 != 7);` la variabile `r` vale **falso** (0).

Attenzione

L'errore più comune del C: confondere `=` (assegnazione, "prende") con `==` (confronto, "doppio uguale"). Scrivere `if (x = 5)` **non confronta**: **assegna** 5 a `x`, e la condizione risulta sempre vera.

5.4 Operatori logici

Servono a **combinare più condizioni**, con la stessa logica delle porte logiche:

Simbolo	Nome	È vero quando...
&&	AND (e)	**tutte e due** le condizioni sono vere
||	OR (o)	**almeno una** delle condizioni è vera
!	NOT (non)	la condizione è **falsa**

Esempio

`if (temperatura > 18 && temperatura < 25)`: vero solo se la temperatura è tra 18 e 25. `if (tasto == 'x' || tasto == 'X')`: vero se si è premuto x minuscolo *oppure* maiuscolo.

6 Prendere decisioni: `if` e `if ... else`

Definizione

`if` esegue un blocco di codice **solo se** una condizione è **vera**; se è falsa, il blocco viene **saltato**.

```
if (CONDIZIONE)
{
  // eseguito solo se CONDIZIONE e' vera
}
```

Con **`else`** si dice anche cosa fare quando la condizione è **falsa**: il programma esegue sempre **uno dei due** blocchi.

```
if (CONDIZIONE)
{
  // se e' vera
}
else
{
  // se e' falsa
}
```

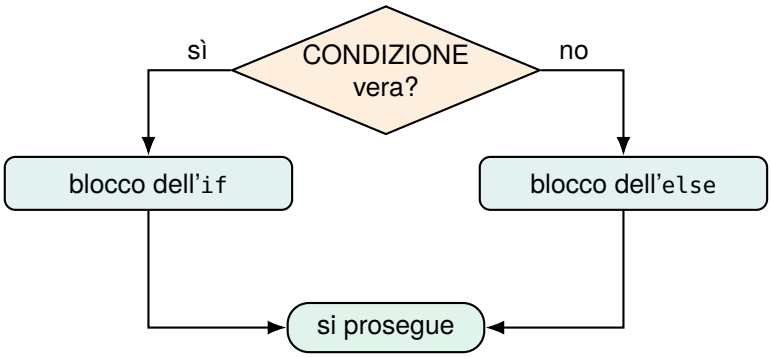


Figura 2: Il diagramma di flusso di `if ... else`: si passa sempre da uno solo dei due blocchi. Senza `else`, il ramo "no" va direttamente avanti.

Esempio

```
int a = 9;
int b = 7;
if (a < b)
{
  Serial.println("a e' minore di b");
}
else
{
  Serial.println("a non e' minore di b");
}
```

La condizione `a < b` (9 < 7) è **falsa**, quindi viene eseguito il blocco dell'`else`. Con `a = 5` la condizione sarebbe vera e verrebbe stampato il primo messaggio.

Nota

Se nel blocco c'è **una sola istruzione** le graffe non sono obbligatorie, ma è buona norma **metterle sempre**: il codice è più chiaro e si evitano errori. Si possono scrivere più `if` uno dopo l'altro: ognuno viene controllato **separatamente**, e possono risultare veri anche più di uno.

7 Ripetere: `for`, `while`, `do ... while`

7.1 Il ciclo `for`

Definizione

Il **`for`** ripete un blocco di codice un **numero determinato** di volte:

```
for (partenza; condizione; incremento)
{
  // codice da ripetere
}
```

- **partenza**: eseguita **una volta sola**, all'inizio; di solito si dà un valore iniziale alla variabile

che conta le ripetizioni;

- **condizione**: controllata **prima di ogni ripetizione**; se è vera si esegue il blocco, se è falsa il ciclo finisce;
- **incremento**: eseguito **alla fine di ogni ripetizione**; modifica la variabile, così prima o poi la condizione diventa falsa.

Esempio

```
for (int i = 0; i < 4; i++)
{
  // codice ripetuto 4 volte
}
```

`i++` significa `i = i + 1`. La variabile `i` vale 0, 1, 2, 3: il blocco viene eseguito **4 volte**. Quando `i` diventa 4 la condizione `i < 4` è falsa e il ciclo termina. Si può anche contare all'indietro (`i--`) o a passi diversi (`i = i + 2`).

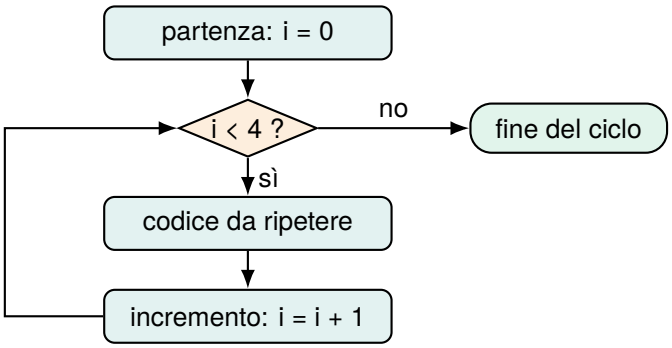


Figura 3: Il diagramma di flusso del ciclo `for`.

7.2 Il ciclo `while`

Definizione

Il **`while`** ("mentre") ripete un blocco **finché** una condizione resta vera. La condizione viene controllata **prima** di ogni ripetizione: se è falsa già all'inizio, il blocco non viene eseguito neanche una volta.

```
int contatore = 1;            // partenza
while (contatore <= 5)        // condizione
{
  Serial.println(contatore);
  contatore = contatore + 1;  // incremento: se manca, il ciclo non finisce mai
}
```

Il programma stampa i numeri da 1 a 5.

Attenzione

Ciclo infinito: se ci si dimentica di aggiornare la variabile della condizione, la condizione resta sempre vera e il ciclo **non termina mai**.

Nota

Esiste anche il **`do ... while`**, che controlla la condizione **alla fine**: il blocco viene eseguito **almeno una volta**. Attenzione al punto e virgola dopo la condizione: `do { ... } while (condizione);`

Definizione

`for` o `while`? Tutto quello che si fa con uno si può fare con l'altro: la scelta è una questione di **stile**. Di solito si usa il **`for`** quando il numero di ripetizioni è **noto prima** ("accendi il LED 10 volte"), il **`while`** quando dipende da **eventi esterni** e si scopre solo *durante* l'esecuzione ("aspetta finché il pulsante non viene premuto").

8 Ingressi e uscite digitali

Esempio

Le porte del supermercato. Al supermercato da alcune porte si entra e da altre si esce. Anche i pin di Arduino sono "porte": da alcuni le informazioni **entrano** (**input**), da altri **escono** (**output**). Su un'uscita è Arduino a decidere: acceso o spento. Su un'entrata decide il mondo esterno, e Arduino ascolta. Come per noi la **bocca** (per parlare) e le **orecchie** (per ascoltare): non si parla con le orecchie.

Definizione

- Un **attuatore** trasforma una grandezza elettrica in un'altra grandezza fisica (un LED la trasforma in luce, un motore in movimento): si collega a un pin di **output**.
- Un **sensore** trasforma una grandezza fisica in un'altra, di solito in una **tensione** o in una **corrente** che Arduino può misurare (un pulsante, un sensore di temperatura): si collega a un pin di **input**.

Le informazioni che arrivano possono essere **digitali** (solo due stati: acceso/spento, vero/falso, HIGH/LOW) oppure **analogiche** (un valore qualsiasi in un intervallo).

8.1 I comandi

- **`pinMode(pin, modo)`**: decide se il pin è un ingresso (`INPUT`) o un'uscita (`OUTPUT`). Si scrive di solito in `setup()`. Esempi: `pinMode(3, OUTPUT);` per un LED, `pinMode(7, INPUT);` per un pulsante.
- **`digitalWrite(pin, stato)`**: porta un pin di uscita a 5 V (`HIGH`, il LED si **accende**) oppure a 0 V (`LOW`, il LED si **spegne**).

- **`digitalRead(pin)`**: legge un pin di ingresso e dà `HIGH` o `LOW`. Il valore va messo in una variabile: `int stato = digitalRead(7);`
- **`delay(ms)`**: ferma il programma per il numero di **millisecondi** indicato (`delay(1000)` = un secondo). Accetta solo numeri interi; per tempi più brevi c'è `delayMicroseconds(us)`, in microsecondi.

9 Il pulsante e le resistenze di pull-up e pull-down

Definizione

Il **pulsante** è il sensore digitale più semplice. Si comporta in due modi: **circuito aperto** (non passa corrente) oppure **cortocircuito** (passa corrente), a seconda che sia premuto o no. In commercio ce ne sono infiniti modelli, ma il simbolo negli schemi è sempre lo stesso.

Attenzione

Un pin di ingresso collegato **solo** al pulsante, quando il pulsante è aperto, non è collegato a niente: si dice che è "**flottante**", e legge valori a caso (basta avvicinare la mano). Per questo il pin va sempre "tirato" a un livello preciso da una **resistenza**: verso i 5 V (**pull-up**) oppure verso GND (**pull-down**). Il valore tipico è **10 kOhm** (10 kΩ).

9.1 Pulsante con resistenza di pull-up

Con la resistenza di **pull-up**, a pulsante **non premuto** il pin legge **HIGH** (la resistenza lo collega ai 5 V); a pulsante **premuto** legge **LOW** (il pulsante lo collega a GND). La logica è **invertita**: premuto = LOW.

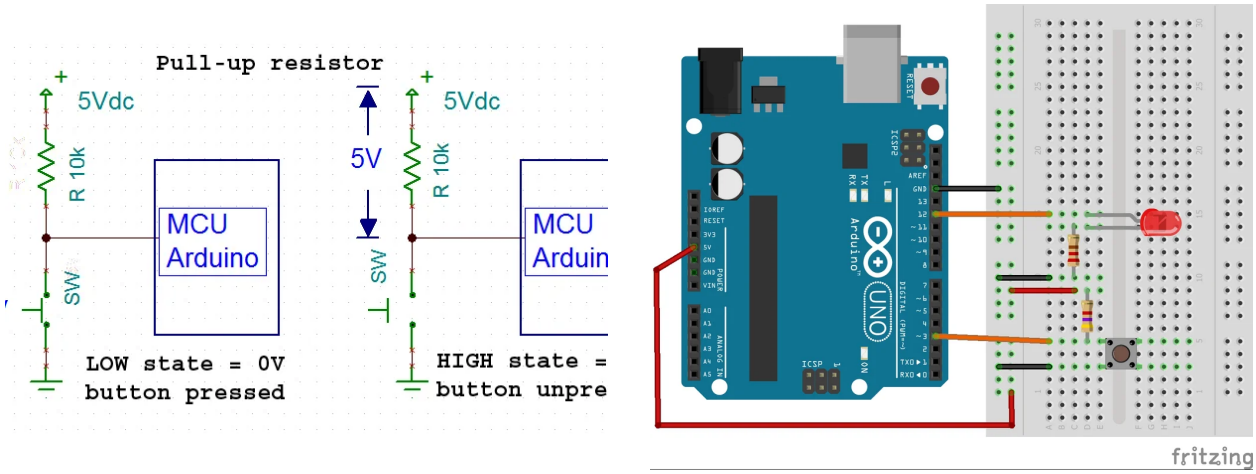


Figura 4: Pull-up. A sinistra lo schema: resistenza da 10 kΩ tra il pin e i 5 V, pulsante tra il pin e GND; premuto = 0 V (LOW), non premuto = 5 V (HIGH). A destra il montaggio su breadboard, con il LED sul pin 12 e il pulsante sul pin 3.

```
/* test della resistenza di pull-up */
void setup()
{
  pinMode(3, INPUT);    // pulsante
  pinMode(12, OUTPUT);  // LED
}

void loop()
{
  int statoPulsante = digitalRead(3);   // legge il pulsante
  if (statoPulsante == LOW)             // pull-up: premuto = LOW
  {
    digitalWrite(12, HIGH);             // accende il LED
  }
  else
  {
    digitalWrite(12, LOW);              // spegne il LED
  }
}
```

Nota

Arduino ha anche una **resistenza di pull-up interna**, che si attiva con `pinMode(3, INPUT_PULLUP);`: in quel caso non serve la resistenza esterna, e la logica è quella del pull-up (premuto = LOW).

9.2 Pulsante con resistenza di pull-down

Con la resistenza di **pull-down**, a pulsante **non premuto** il pin legge **LOW** (la resistenza lo collega a GND); a pulsante **premuto** legge **HIGH** (il pulsante lo collega ai 5 V). La logica è quella “naturale”: premuto = HIGH.

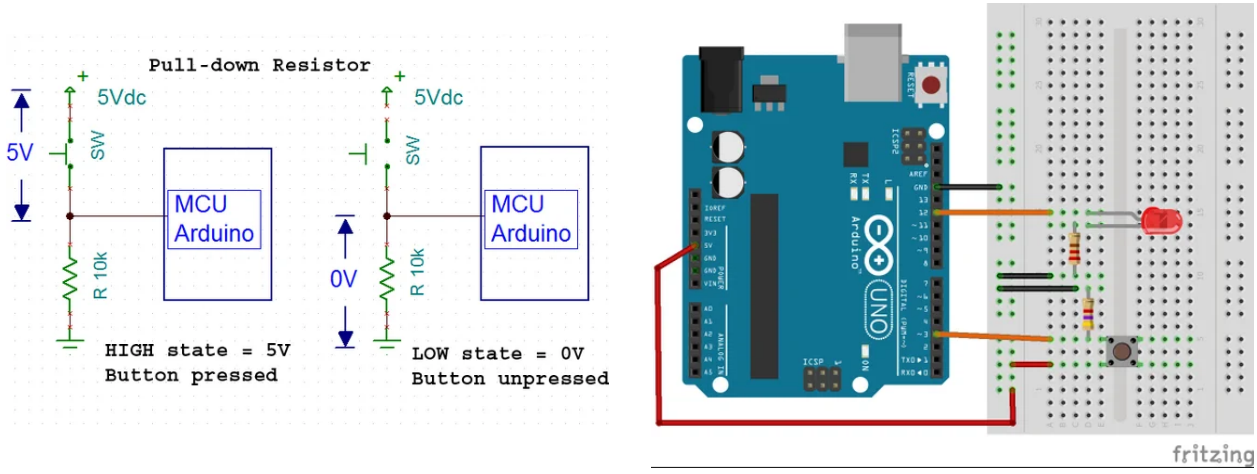


Figura 5: Pull-down. A sinistra lo schema: pulsante tra i 5 V e il pin, resistenza da 10 kΩ tra il pin e GND; premuto = 5 V (HIGH), non premuto = 0 V (LOW). A destra il montaggio su breadboard.

```
/* test della resistenza di pull-down */
int pinPulsante = 3;
int pinLed = 12;

void setup()
{
  pinMode(pinPulsante, INPUT);
  pinMode(pinLed, OUTPUT);
}

void loop()
{
  int statoPulsante = digitalRead(pinPulsante);
  if (statoPulsante == HIGH)   // pull-down: premuto = HIGH
  {
    digitalWrite(pinLed, HIGH);
  }
  else
  {
    digitalWrite(pinLed, LOW);
  }
}
```

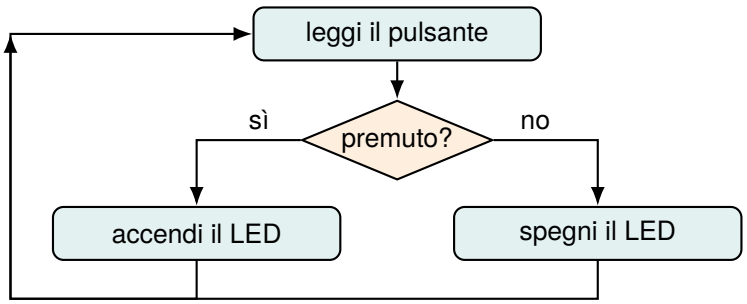


Figura 6: Il diagramma di flusso dei due programmi del pulsante. Cambia solo che cosa vuol dire “premuto”: LOW con il pull-up, HIGH con il pull-down.

10 La comunicazione seriale

Il **monitor seriale** è uno strumento dell’ambiente Arduino che mostra il testo che Arduino invia, e permette di inviare testo ad Arduino. Si apre dal menu *Strumenti* → *Monitor seriale*, oppure con l’icona a lente in alto a destra.

10.1 Inizializzare: `Serial.begin`

Definizione

Prima di usare la seriale bisogna **inizializzarla**, una volta sola, quindi in `setup()`:

```
Serial.begin(9600);
```

`Serial` è la **classe** che gestisce la comunicazione, **`begin`** è il **metodo** che la imposta. Il numero è il **baud rate**, la **velocità** di comunicazione: più è alto, più veloce è il trasferimento. Valori possibili: 300, 600, 1200, 2400, 4800, **9600**, 14400, 19200, 28800, 31250, 38400, 57600, **115200**. I più usati sono 9600 e 115200.

Attenzione

Nel monitor seriale, in basso a destra, c'è un menu con la **velocità di ricezione**: deve essere **uguale** a quella scritta in `Serial.begin`, altrimenti si leggono caratteri senza senso. Non succede in automatico: lo dovete impostare voi.

10.2 Trasmettere: `Serial.print` e `Serial.println`

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  Serial.print("Ciao a tutti");
  delay(1000);   // per non intasare il canale con troppi messaggi
}
```

`Serial.print` scrive sul monitor **senza andare a capo**: i messaggi si mettono uno dietro l'altro sulla stessa riga. **`Serial.println`** scrive e poi **va a capo**.

Attenzione

Se copiate del codice da un documento (Google Documenti, Word), le **virgolette** possono essere quelle "tipografiche" (“ ”), che per Arduino sono un **carattere diverso** e danno errore di compilazione. Nel codice vanno le virgolette **diritte** (" "): meglio riscriverlo a mano.

10.3 Ricevere: `Serial.available` e `Serial.read`

Esempio

Lo sportello e la coda. I dati che arrivano dal computer finiscono in una memoria chiamata **buffer**, come le persone in **coda** davanti a uno sportello. **`Serial.available()`** dice **quanti** dati sono in coda (0 se non è arrivato niente). **`Serial.read()`** fa passare allo sportello il **primo** della coda: lo **legge** e lo **toglie** dalla coda.

Definizione

Prima si controlla **se** è arrivato qualcosa, poi lo si legge e si decide cosa farne. Come `digitalRead`, i due comandi **danno un risultato**, che si mette in una variabile:

```
int x = Serial.available();   // quanti dati in coda?
if (x != 0)                   // e' arrivato qualcosa
{
  int b = Serial.read();      // leggo il primo e lo tolgo dalla coda
  // ... decido cosa fare in base a b
}
```

Esempio

Un LED diverso per ogni lettera. In base alla lettera ricevuta (`c`, `f` oppure `u`) si accende un LED diverso, sui pin 2, 3 e 4:

```
void setup()
{
  Serial.begin(9600);
  pinMode(2, OUTPUT);   // LED 1
  pinMode(3, OUTPUT);   // LED 2
  pinMode(4, OUTPUT);   // LED 3
}

void loop()
{
  int x = Serial.available();
  if (x != 0)
  {
    int b = Serial.read();
    if (b == 'c')
    {
      digitalWrite(2, HIGH);
    }
    if (b == 'f')
    {
      digitalWrite(3, HIGH);
    }
    if (b == 'u')
    {
      digitalWrite(4, HIGH);
    }
  }
}
```

Una volta accesi, i LED restano accesi: per spegnerli servirebbe un'altra lettera, con un altro `if` e `digitalWrite(..., LOW)`. Un altro esempio, quello fatto in classe il 15/10 ("ciao" e "premuto"), è nella dispensa A01.

11 Esercizi per ripassare

Esempio

1. Quanti byte occupa un `char` su Arduino Uno? E un `int`? Che intervallo di valori ha ciascuno?
2. Converti in decimale il numero binario 1011, e in binario il numero decimale 13.
3. Quanto valgono `17 / 5` e `17 % 5` tra numeri interi?
4. Scrivi un programma che faccia lampeggiare un LED sul pin 8 per 10 volte, con un `for`, e poi si fermi (suggerimento: il `for` va in `setup()`).
5. Collega un pulsante con resistenza di pull-down al pin 2: finché è premuto, il LED sul pin 13 deve restare acceso.
6. Con la seriale: quando arriva la lettera `a` il LED si accende, quando arriva `s` si spegne. Ogni volta Arduino risponde sul monitor con “acceso” o “spento”.
7. Trova l’errore: `if (x = 3) { digitalWrite(5, HIGH); }`

12 Cosa ricordare

- Arduino Uno: USB (alimentazione e dati), presa per jack, **GND**, **5V**, pin digitali **0–13** (0 e 1 per la seriale), ingressi analogici **A0–A5**; microcontrollore **ATmega328**.
- **`setup()`** una volta, **`loop()`** all’infinito; in C “normale” parte tutto da `main()`. Punto e virgola, graffe, commenti, maiuscole e minuscole.
- 1 byte = 8 bit = 256 valori. `char` 1 byte ($-128 \ldots 127$), `byte` $0 \ldots 255$, `int` 2 byte, `float` 4 byte con la virgola. Array: indici da 0 a dimensione-1.
- Operatori: `=` assegna; `+ - * / %` (la `/` tra interi dà il quoziente, la `%` il resto); confronti `> < >= <= == !=`; logici `&& || !`. Mai `=` al posto di `==`.
- `if` / `else` per decidere; `for` se le ripetizioni sono note, `while` se dipendono da eventi esterni.
- `pinMode`, `digitalWrite`, `digitalRead`, `delay`. Attuatori sulle uscite, sensori sugli ingressi.
- Pulsante: sempre con **pull-up** (premuto = LOW) o **pull-down** (premuto = HIGH), 10 kΩ; oppure `INPUT_PULLUP`.
- Seriale: `Serial.begin` (stessa velocità nel monitor), `print`/`println`, `available` (quanti in coda), `read` (legge e toglie dalla coda).